SUPERMARKET MANAGEMENT WEBSITE

Name	ID
David Gerges	192300429
David Alber	192300384
Martin Milad	192300344
Mina Ehab	192300365
Nicole Aziz	192300428
Nour Romany	192300393
Seif Eldeen Waleed	192300230

DR. AHMED OSMAN

ENG. ABDULRAHMAN GABER

ENG. ASMAA ELBOGDADY

Table of contents

System Overview 2
System Functionalities 2
USE CASE DIAGRAM 3
CLASS DIAGRAM 5
SEQUENCE DIAGRAM 6
CONTEXT DIAGRAM 7
SYSTEM FLOWCHART 8
GRAPH 9
CYCLOMATIC COMPLEXITY 10
UFC 10
FUNCTION POINTS 11
COAST FUNCTION 13
COAST ESTIMATION 13
SUMMARY 13

System Overview

The Supermarket Management Website is an online system that digitizes supermarket operations for both customers and administrators. It was created to replace manual processes with a faster, more organized, and accessible platform. Customers can browse products, manage their carts, and place orders, while admins can control inventory, products, and user activity. The project aims to improve convenience, reduce errors, and provide a scalable solution that demonstrates practical software engineering principles.

System Functionalities

Customer Functions

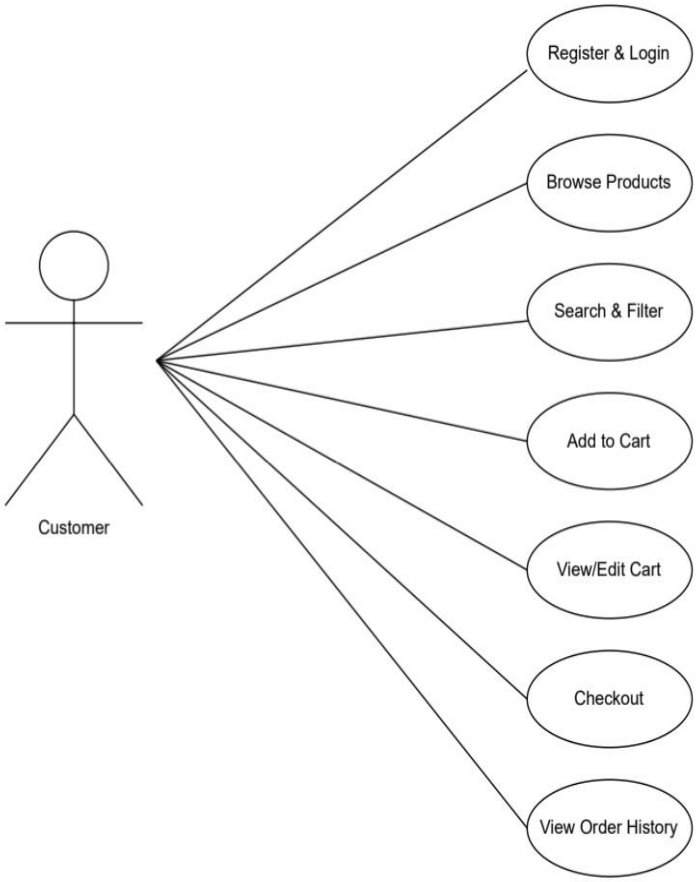
Functionality	Description
User Registration & Login	Allows customers to create accounts and log in securely to access personalized services.
Browse Products	Displays all available products organized into categories such as groceries, drinks, and snacks.
Search & Filter Products	Enables customers to quickly find products by name, price range, or category.
Add to Cart	Let users add selected items to their virtual shopping cart for later checkout.
View & Edit Cart	Provides a detailed view of all items in the cart, with the ability to modify quantities or remove products.
Checkout & Payment Simulation	Finalizes the purchase process by confirming orders and simulating payment.
Order History	Allows users to view previous orders, including statuses such as pending or delivered.

Admin Functions

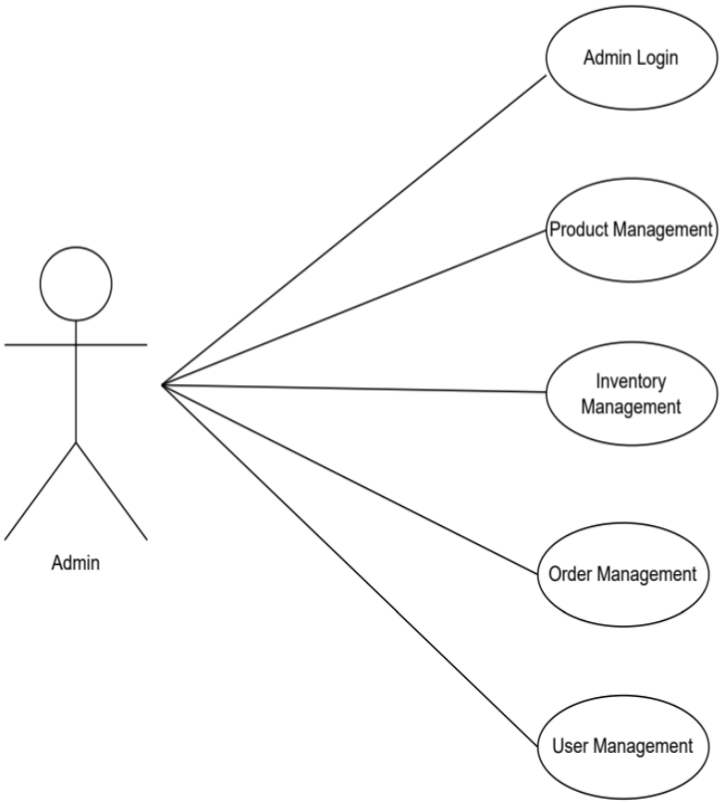
Functionality	Description
Admin Login	Ensures secure access to the administrative dashboard.
Product Management	Enables admins to add new products, update existing ones, or delete outdated items.
Inventory Management	Allows tracking of stock levels and updating quantities when restocking is needed.
Order Management	Let admins view, update, and manage customer orders and their statuses.
User Management	Provides access to a list of registered users and tools to manage their information.

USE CASE DIAGRAM

The customer interacts with the system to perform essential shopping activities. They can register or log in, browse and search for products, add items to their cart, and proceed to checkout. The customer can also view and manage their cart, as well as check their order history. These actions represent the complete online shopping workflow.



The admin uses the system to manage and control supermarket operations. They log in to access tools for adding or updating products, managing inventory levels, handling customer orders, and overseeing registered users. These functionalities allow the admin to keep the system organized, accurate, and up to date.



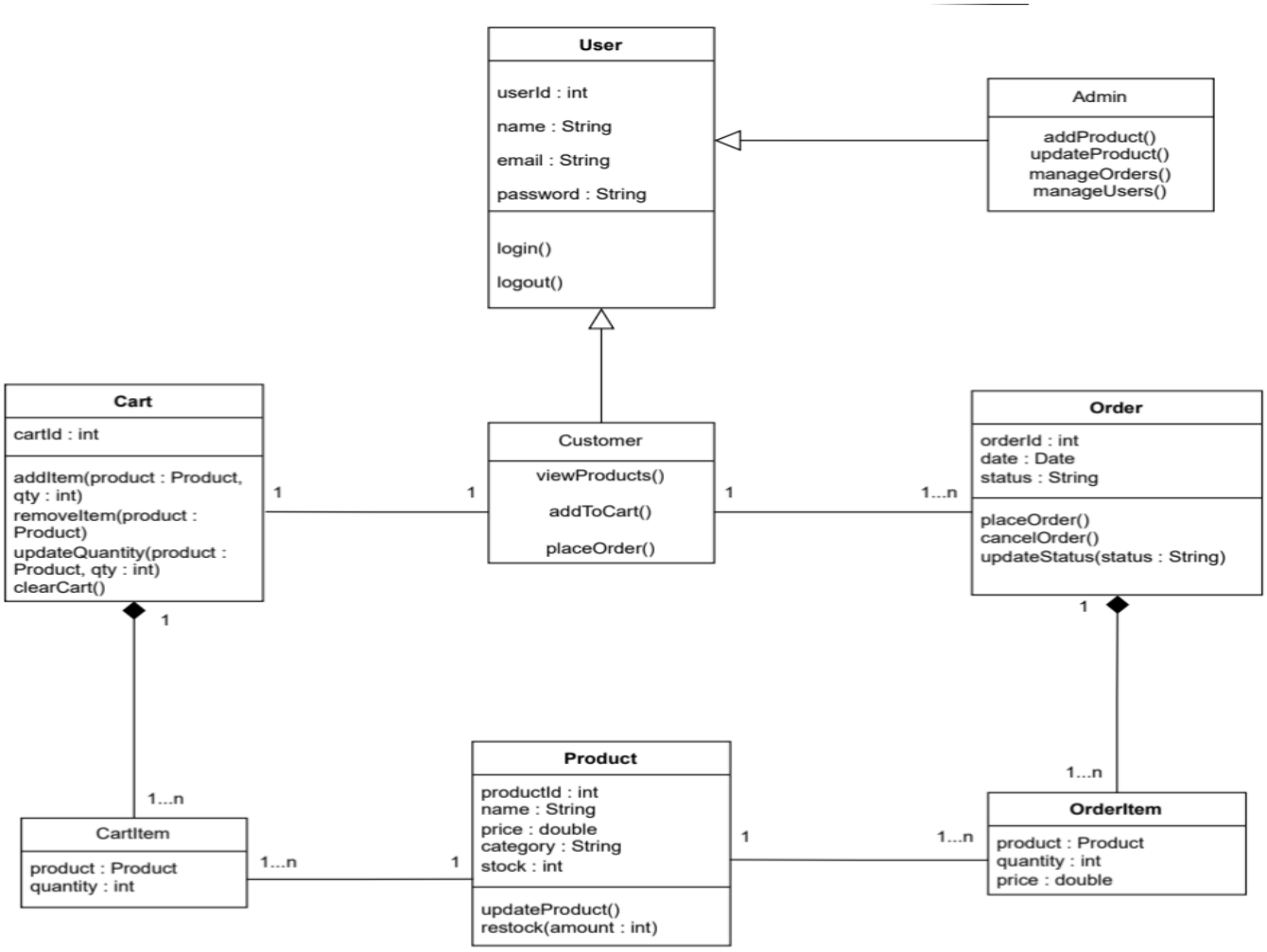
CLASS DIAGRAM

The class diagram shows the main structural components of the Supermarket Management System and how they interact. The system includes a **User** superclass, which is extended by **Customer** and **Admin**. Customers use the system to shop, while admins manage products.

A **Product** represents an item sold in the supermarket. Customers have a **Cart**, which contains multiple **CartItems**, each linking to a specific product and quantity. When a customer completes a purchase, an **Order** is created. Each order contains several **OrderItems**, which store the product details and quantity purchased.

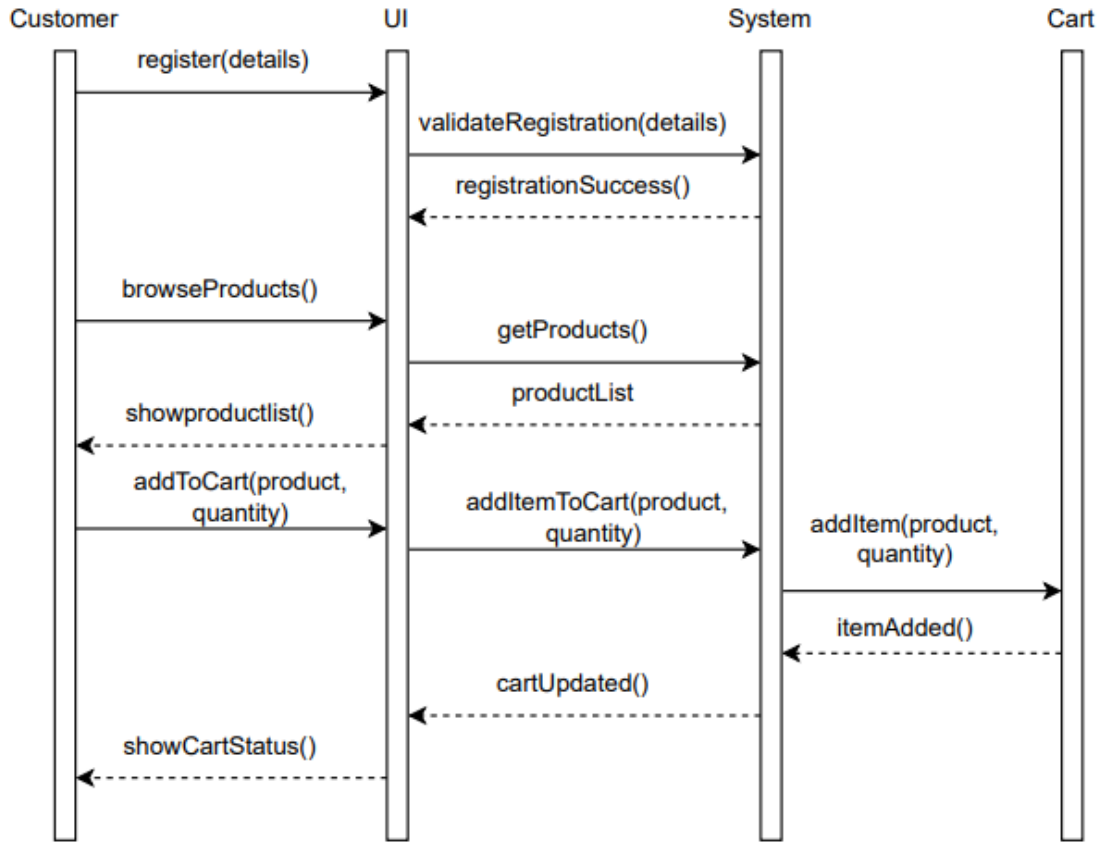
The key relationships include:

- **Inheritance:** Customer and Admin inherit from User.
- **Composition:** Cart contains CartItems; Order contains OrderItems.
- **Associations:** Customers have one Cart and multiple Orders; CartItems and OrderItems each reference one Product.



Sequence diagram

The customer interacts with the UI to register and browse products. The UI forwards all requests to the Supermarket System, which validates the registration and manages system logic. When the customer adds a product to the cart, the system updates the Cart component and returns a confirmation. The UI then displays the updated cart status to the customer, completing the process.



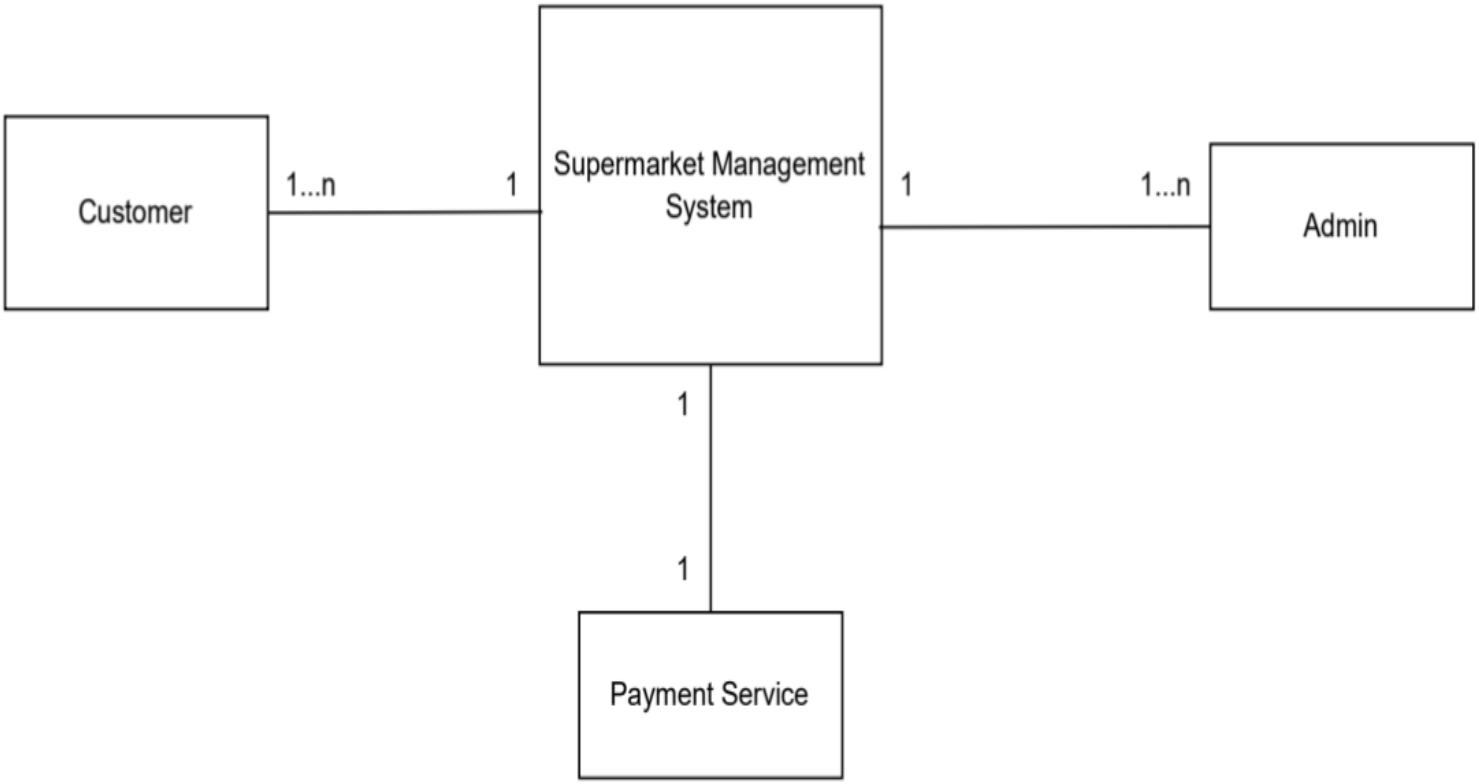
CONTEXT DIAGRAM

- The context diagram shows The **Supermarket Management System** acts as the central system that manages all supermarket operations.

- **Customers** use the system to browse products, place orders, and make payments.

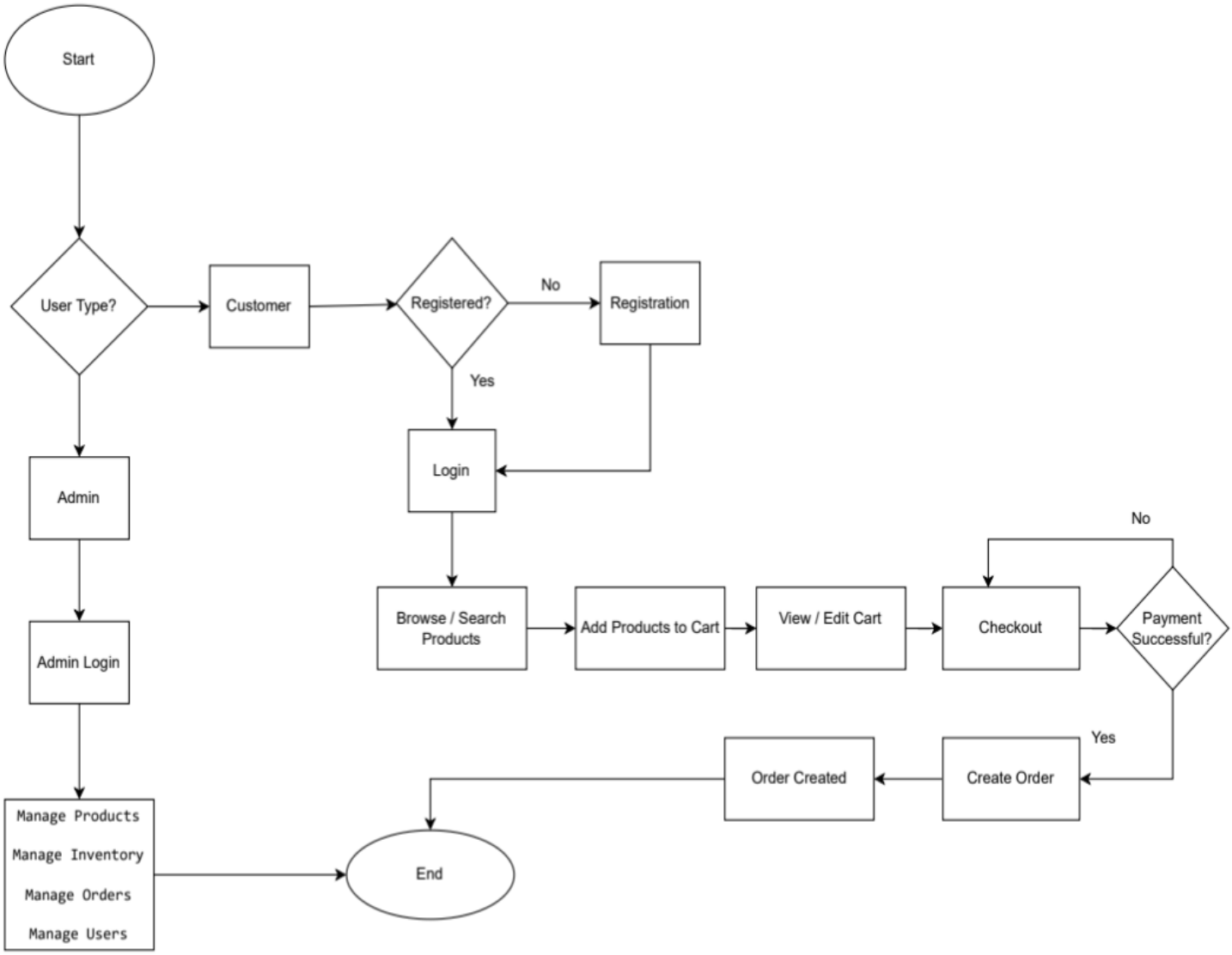
- **Admins** manage products, users, inventory, and orders through the system.

- The **Payment Service** processes payment requests and returns payment confirmations.

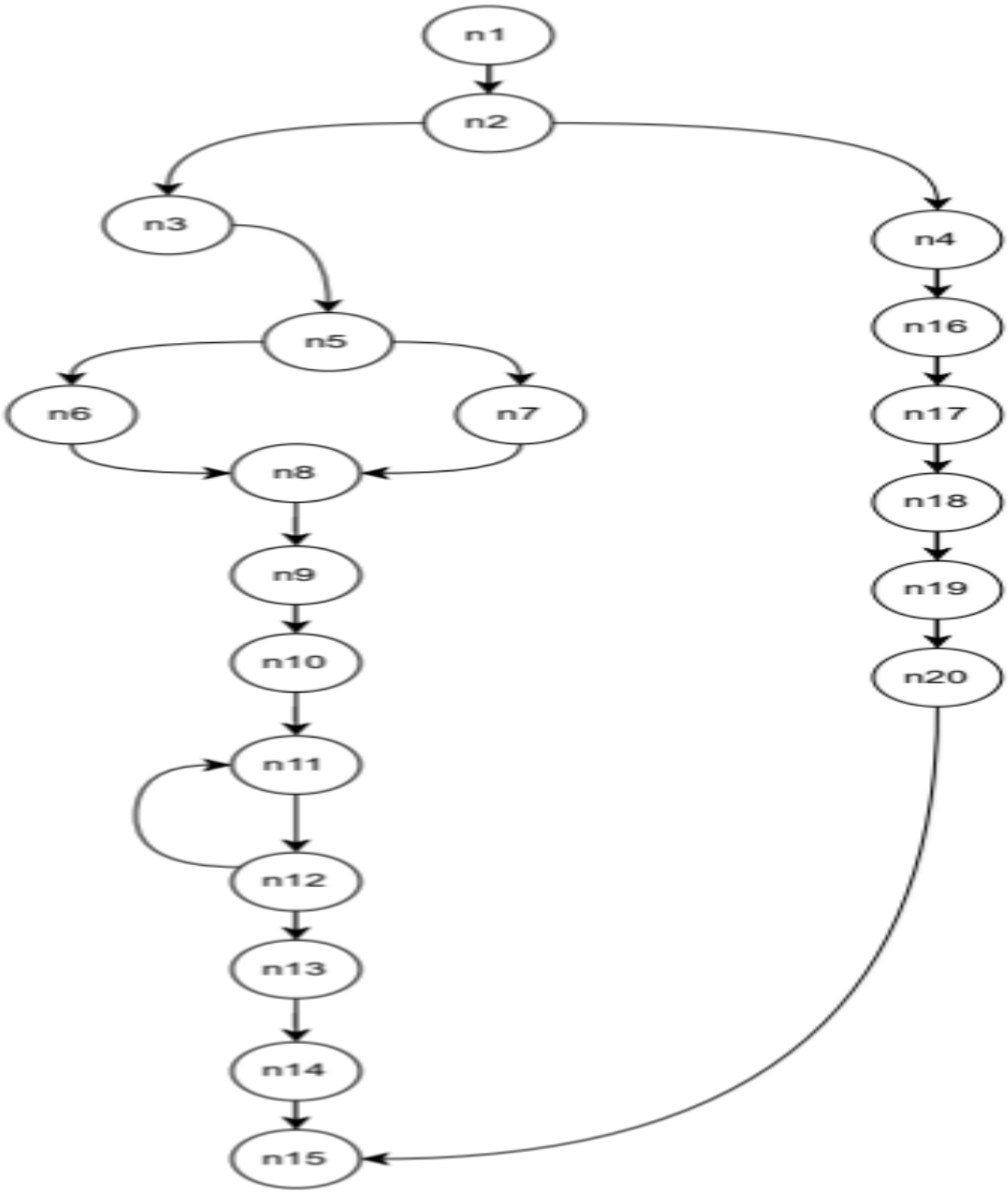


SYSTEM FLOWCHART

The flowchart shows how the Supermarket Management System operates for both customers and admins. Customers register or log in, browse products, add items to the cart, complete checkout, and create an order after successful payment. Admins log in to manage products, inventory, users, and orders, after which the process ends.



GRAPH


n1
n2
n3
n4
n5
n6
n7
n8
n9
n10
n11
n12
n13
n14
n15
n16
n17
n18
n19
n20

CYCLOMATIC COMPLEXITY

Cyclomatic Complexity was calculated to measure the **logical complexity** of the system by identifying the number of **independent execution paths** created by decision statements, loops, and conditional logic in the code. It helps evaluate the system's **testability, maintainability, and risk level**.

CC = 12

This result indicates that the system has **moderate to slightly high complexity** and requires **at least 12 test cases** for complete path coverage. The program remains **testable and manageable**, but special attention is needed when testing critical functions such as authentication, checkout, and admin operations.

UFC

The system functions were identified from the **user's perspective** and counted based on inputs, outputs, interactions, and data storage. This count represents the system's **functional size**, showing that it supports multiple features and user roles, and reflects the level of effort required for development and testing.

UFC	No.	Weight	Total
External Inputs	6	4	24
External Outputs	5	5	25
User Interactions	4	4	16
Logical Internal Files	4	10	40
External Interfaces	1	7	7
Total UFC			112

FUNCTION POINTS

Explain how function points were calculated and what they measure. Describe what the final value indicates about the overall functionality and complexity of the system.

Fi	No. & Complexity Rating
Backup and Recovery	***Rating: 3*** Basic data persistence and recovery are supported using browser storage, but advanced backup mechanisms are not implemented.
Data Communication	***Rating: 3*** The system communicates data between multiple pages and browser storage (localStorage), enabling data sharing across modules.
Distributed Processing Functions	***Rating: 2*** Processing is mainly client-side, with limited distribution between UI and storage layers.
Is Performance Critical?	***Rating: 3*** Performance is important for user experience during product browsing and checkout, but real-time constraints are not strict.
Existing Operating Environment	***Rating: 3*** The system runs in standard web browsers without special hardware or operating system constraints.
On-line Data Entry	***Rating: 4*** Users frequently enter data online, including login credentials, product details, cart updates, and order information.
Input Transaction Built Over Multiple Screens	***Rating: 4*** Key operations such as registration, checkout, and product management involve multiple screens and steps.
Master Files Updated Online	***Rating: 4***

	Core data files (users, products, orders) are updated dynamically during system operation.
Complexity of Inputs, Outputs, Files, and Application Designed for User-Facilitated Changes	***Rating: 3*** The system has moderate complexity in handling various inputs and outputs with some user-driven changes.
Complexity of Processing	***Rating: 3*** Processing includes validation, stock checking, order creation, and status updates, which are moderately complex.
Code Designed for Re-use	***Rating: 2*** Some functions are reusable, but the system is not heavily optimized for reuse across different applications.
Are Conversion/Installation Included in Design?	**Rating: 1** Minimal installation is required; the system runs directly in a browser without data migration or conversion.
Multiple Installations	***Rating: 2*** The system can be deployed on multiple machines via browsers, but without complex deployment requirements.
Application Designed for User-Facilitated Changes	***Rating: 4*** Admins can easily manage products, inventory, and orders through the interface.

$$FP = UFC \times \left[0.65 + 0.01 \times \sum_{i=1}^{14} F_i\right]$$

UFC=112

Summation Fi=41

FP=112x[0.65+0.01x41]=112x[1.06]=118.72

COAST FUNCTION

The cost estimation for the SuperMart system is based on a **Function Point–Based cost model**. This approach estimates project cost using the system's **functional size**, measured in Function Points, rather than lines of code. It focuses on the **functionality delivered to users**, making it well-suited for web-based applications.

The main factors considered in this cost model include:

- Total Function Points of the system
- Average development effort per Function Point
- System complexity and functionality
- Development environment and tools

These factors help convert functional requirements into an estimated development effort and overall project cost.

COAST ESTIMATION

The total development cost was estimated by multiplying the **calculated Function Points** of SuperMart by an assumed **average cost per Function Point**. This estimation assumes:

- A **moderate-complexity web application**
- A **small development team**
- Standard development tools and technologies
- No real-time or safety-critical constraints

The estimation results indicate that SuperMart can be developed within a **reasonable budget and timeframe**, making the project **feasible** for academic or small-scale commercial implementation.

SUMMARY

This document presented an analysis of the **SuperMart supermarket management system**, a web-based application designed to support both customers and administrators. Customers can browse products, manage carts, place orders, and track purchases, while administrators manage products, inventory, users, and orders.

Software engineering metrics such as **Cyclomatic Complexity**, **Function Points**, and **Function Point–Based cost estimation** were used to evaluate the system's complexity, size, and development effort. The analysis shows that SuperMart has **moderate complexity**, well-defined functionality, and is **testable, maintainable, and feasible to develop**.